CURSO · DÍA 1 DE 3

Bienvenido a Docker

Fundamentos y comandos esenciales para equipos .NET



Jorge López Blasco · Formador IT

Antes de empezar

JL Quién os habla

Formador y consultor IT. Matemático de formación, con experiencia en proyectos de datos y desarrollo en banca y consultoría antes de dedicarme a la formación técnica.

? Quiénes sois

Ronda rápida: nombre, en qué aplicaciones trabajáis y si habéis tenido algún contacto con Docker, aunque sea de oídas.

3 Cómo trabajaremos

Teoría corta, demostración y laboratorio. Preguntad en cualquier momento: las dudas de uno suelen ser las de todos. La sesión queda grabada.

Qué vamos a conseguir en tres días

1 Día 1 · Fundamentos

Qué es un contenedor y qué problemas resuelve. Arquitectura de Docker. Ciclo de vida y comandos esenciales. Imágenes, capas y registros.

2 Día 2 · Docker y .NET

El Dockerfile. Contenerizar una API ASP.NET Core con construcción multi-etapa. Configuración por entorno. Qué pasa con .NET Framework.

3 Día 3 · Aplicaciones completas

Volúmenes, redes y Docker Compose. Buenas prácticas. Qué infraestructura hace falta para pasar de IIS a contenedores.

Agenda de hoy

09:00	Bienvenida, presentaciones y objetivos
09:20	¿Por qué contenedores?
09:55	Contenedores y Docker: conceptos y arquitectura
10:30	**Lab 01 · Primeros pasos**
11:00	Descanso
11:20	Ciclo de vida y comandos esenciales
11:50	**Lab 02 · Ciclo de vida de un contenedor**
12:40	Imágenes, capas y registros
13:10	**Lab 03 · Imágenes y registros**
13:50	Resumen del día y preguntas

01

¿Por qué contenedores?

Los problemas que resuelven en el día a día de un equipo de desarrollo



¿Os suena alguna de estas situaciones?

1 «En mi máquina funciona»

La aplicación va bien en desarrollo y falla en preproducción o en producción por una versión distinta de un componente, una configuración o una dependencia instalada a mano.

2 Despliegues artesanales

Preparar el servidor, instalar el runtime, configurar el sitio y el pool de IIS, copiar ficheros, transformar el web.config… y cruzar los dedos en cada paso a producción.

3 Servidores con historia

Cada servidor acumula cambios manuales que nadie documentó. Actualizar una dependencia para una aplicación puede romper otra que comparte el mismo servidor.

4 Incorporaciones lentas

Un compañero nuevo tarda días en tener el entorno funcionando: versiones de SDK, SQL Server local, certificados, variables, rutas…

La idea: estandarizar el envío

Antes de 1956

Cada mercancía se cargaba a mano, con su embalaje, su forma y su manera de estibarla. Lento, caro y con roturas.



Con el contenedor estándar

Mismo formato para todo. Al barco, al tren o al camión le da igual lo que haya dentro.



Docker hace lo mismo con el software: la aplicación viaja con todo lo que necesita, en un formato que cualquier servidor con Docker sabe ejecutar.

Qué ganáis en el día a día

1 Entornos idénticos

Lo que se prueba en preproducción es, bit a bit, lo que se ejecuta en producción. Adiós al «en mi máquina funciona».

2 Despliegue y marcha atrás

Desplegar es arrancar una imagen nueva. Volver atrás es arrancar la anterior, que sigue intacta.

3 Versiones que conviven

.NET 8 y .NET 10 en el mismo servidor sin interferir. Migrar una aplicación no obliga a tocar las demás.

4 Entorno en minutos

Un compañero nuevo levanta la aplicación con su base de datos con un solo comando.

5 Todo documentado en código

El Dockerfile y el compose.yaml dicen exactamente cómo se construye y ejecuta cada aplicación, y se revisan en Git.

6 Encaja con la migración

El .NET moderno está pensado para contenedores. Migrar a .NET y contenerizar se refuerzan mutuamente.

Lo que Docker no es

Ideas que suelen frenar

- «Es una máquina virtual más ligera»
- «Hay que reescribir las aplicaciones»
- «Implica montar Kubernetes»
- «Es una moda que pasará»
- «Es solo para Linux»

Lo que veremos en el curso

- Es un proceso aislado que comparte el núcleo del sistema: arranca en segundos
- El código no cambia; lo que cambia es cómo se empaqueta y se despliega
- Se puede empezar con Docker en un servidor y Compose; el orquestador llega si hace falta
- Lleva más de diez años siendo el estándar de empaquetado del sector
- Hay contenedores Windows, aunque el ecosistema principal es Linux

02

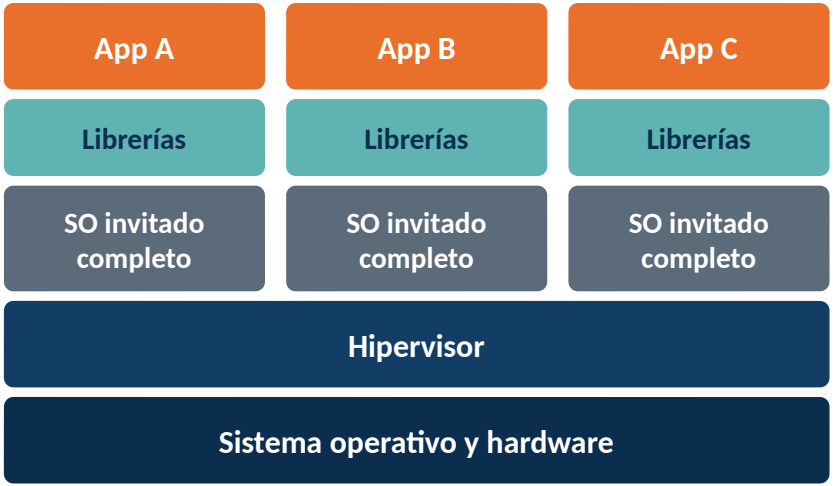
Contenedores y Docker

Conceptos y arquitectura



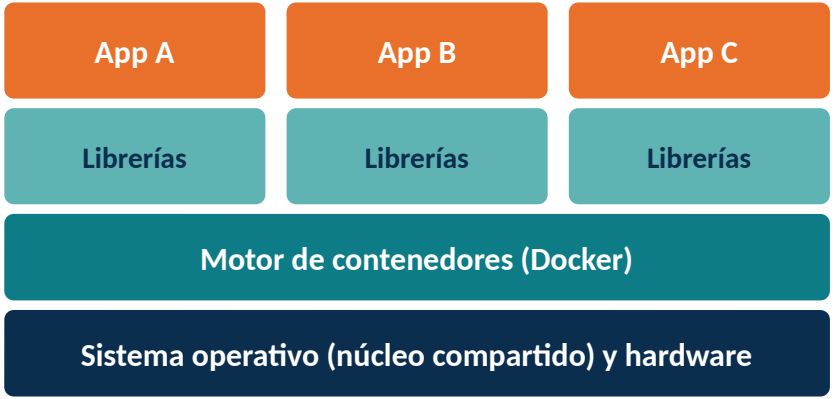
Máquinas virtuales frente a contenedores

Máquinas virtuales



Minutos en arrancar · GB por máquina · aislamiento total

Contenedores



Segundos en arrancar · MB por contenedor · aislamiento de procesos

Cómo está organizado Docker

Cliente

El comando docker que escribís. Envía órdenes al motor a través de una API.

docker run · build · pull

API →

Motor (Docker Engine / daemon)

Imágenes

Plantillas de solo lectura: la aplicación y todo lo que necesita.

Contenedores

Instancias en ejecución de una imagen. Varios contenedores pueden salir de la misma imagen.

pull/push ⇄

Registro

Almacén de imágenes. Docker Hub, mcr.microsoft.com o uno privado.

Docker Desktop

En Windows empaqueta cliente, motor (sobre WSL 2), Compose e interfaz gráfica.

Docker Compose

Describe aplicaciones de varios contenedores en un fichero. Día 3.

Imagen y contenedor: clase y objeto

Imagen ≈ clase

- Plantilla de solo lectura
- Se construye una vez (docker build) o se descarga (docker pull)
- Tiene nombre y versión: convocatorias-api:1.0
- No se ejecuta: describe qué se ejecutará
- Se guarda en un registro y se comparte

→

Contenedor ≈ objeto

- Instancia en ejecución de una imagen
- Se crea con docker run; puede haber muchos de la misma imagen
- Añade una capa de escritura propia y efímera
- Tiene su propio estado, red, procesos y nombre
- Se crea, se para, se elimina y se vuelve a crear

Docker en vuestros equipos Windows

1 Contenedores Linux

Docker Desktop crea una pequeña máquina Linux sobre WSL 2 y ejecuta ahí los contenedores. Es el modo por defecto y el que usaremos en el curso.

2 Contenedores Windows

Docker Desktop puede cambiar a modo Windows para ejecutar imágenes basadas en Windows Server. Necesario para .NET Framework. Lo veremos mañana.

3 Lo que no cambia

Los comandos son los mismos en Windows, Linux y Mac. Lo que aprendáis en el portátil vale tal cual en un servidor.

Primeros pasos

LAB

01

≈ 30 minutos

Comprobar la instalación y arrancar vuestros primeros contenedores.

- docker version y docker info: cliente y motor
- docker run hello-world: qué ocurre por dentro
- Un servidor web nginx en un segundo, sin instalar nada
- docker ps, logs, stop, start y rm
- Un vistazo a Docker Desktop

`labs/lab01-primeros-pasos/README.md`

03

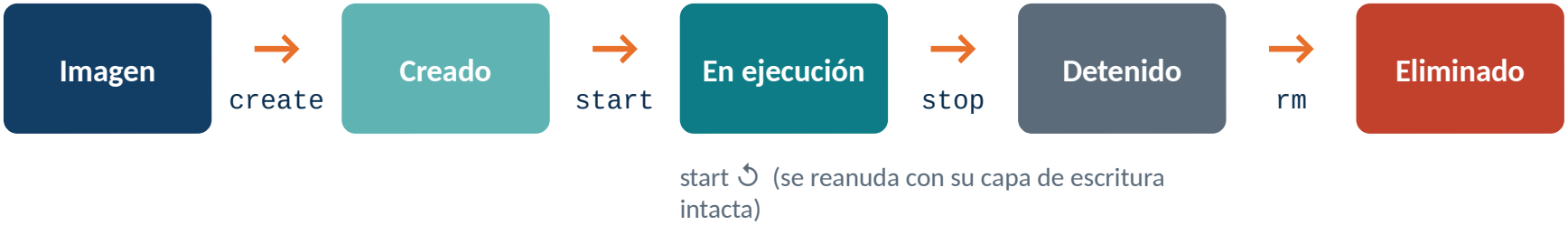
Ciclo de vida y comandos esenciales

Lo que vais a usar el noventa por ciento del tiempo



El ciclo de vida de un contenedor

```
docker run  =  create + start
```



Un contenedor vive mientras vive su proceso principal. Si el proceso termina (hello-world) o falla, el contenedor pasa a detenido. Por eso nginx o una API siguen en ejecución: son servidores que no terminan. Al eliminar un contenedor se pierde su capa de escritura; la imagen no cambia nunca.

Comandos esenciales

Comando	Para qué sirve
`docker run -d -p 8080:80 --name web nginx`	Crear y arrancar en segundo plano, publicando un puerto (equipo:contenedor)
`docker ps` · `docker ps -a`	Contenedores en ejecución · todos, incluidos los detenidos
`docker logs -f web`	Ver y seguir los logs (la salida estándar de la aplicación)
`docker exec -it web sh`	Abrir una terminal dentro de un contenedor en marcha
`docker inspect web`	Toda la configuración en JSON: red, montajes, variables…
`docker stop` · `start` · `rm`	Parar · arrancar · eliminar
`docker run -e CLAVE=valor ...`	Pasar variables de entorno
`docker run --rm -it alpine sh`	Contenedor de usar y tirar: se elimina al salir
`docker run --restart unless-stopped ...`	Reinicio automático, como un servicio de Windows

LA IDEA MÁS IMPORTANTE DE HOY

Un contenedor es desechable.
No se arregla: se reemplaza.

Lo que debe perdurar va en la imagen (código y configuración que se construye) o en un volumen (datos). Nunca en la capa de escritura del contenedor.



Ciclo de vida de un contenedor

LAB

02

≈ 45 minutos

Trabajar con contenedores como en el día a día y comprobar que son efímeros.

- Entrar en un contenedor con exec y modificarlo por dentro
- Comprobar qué sobrevive a stop/start y qué se pierde con rm
- inspect, top, stats y logs -f
- Copiar ficheros con docker cp y pasar variables de entorno
- Contenedores interactivos y políticas de reinicio

`labs/lab02-ciclo-de-vida/README.md`

04

Imágenes, capas y registros

De dónde salen los contenedores y cómo se mueven



Anatomía del nombre de una imagen



Registro

De dónde se descarga. Si se omite: Docker Hub (docker.io).

Repositorio

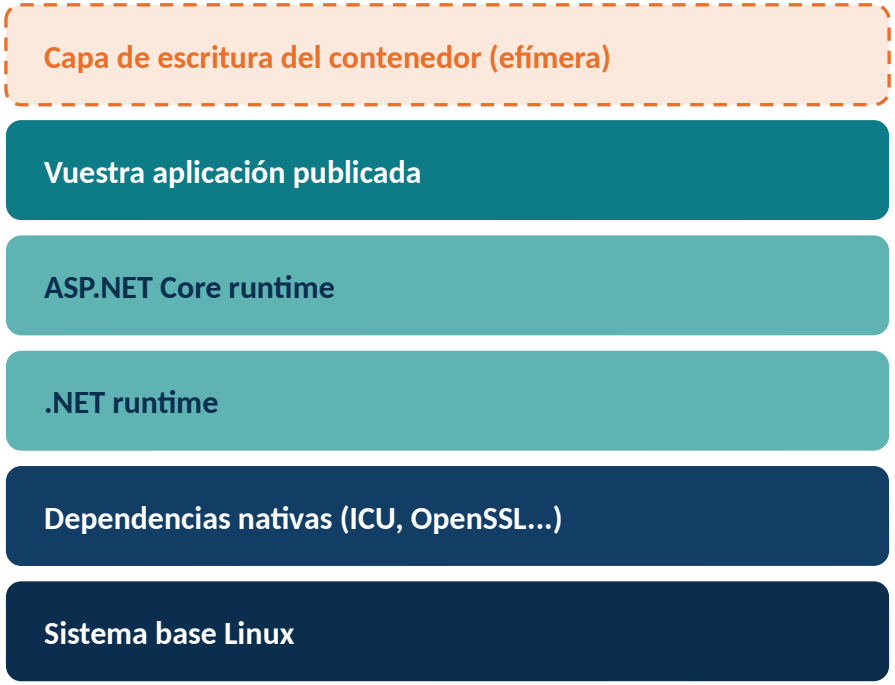
El nombre de la imagen, puede incluir una ruta.

Etiqueta

Versión o variante. Si se omite: latest.

`nginx` → `docker.io/library/nginx:latest`
`nginx:alpine` → misma aplicación, base Alpine Linux, mucho más pequeña
`localhost:5000/convocatorias-api:1.0.0` → vuestra imagen en un registro privado

Las imágenes están hechas de capas



Cada instrucción del Dockerfile genera una capa.

Las capas se comparten: si dos imágenes usan la misma base, se guarda y se descarga una sola vez.

La caché de construcción reutiliza las capas que no han cambiado. Mañana sacaremos partido de esto.

El contenedor añade una capa de escritura que desaparece al eliminarlo.

Registros: de dónde vienen y adónde van las imágenes

1 Registros públicos

Docker Hub para imágenes de la comunidad y de fabricantes (nginx, redis…). mcr.microsoft.com para todo lo de Microsoft: .NET, SQL Server, Windows.

2 Registro privado

Para vuestras imágenes y una copia controlada de las imágenes base. Harbor, Nexus o GitLab, dentro de vuestra red. Lo veremos el día 3.

3 Sin registro: save y load

docker save exporta una imagen a un fichero .tar; docker load la importa en otro equipo. La vía más sencilla para un entorno sin internet.

Imágenes y registros

LAB

03

≈ 40 minutos

Entender cómo están hechas las imágenes y cómo moverlas sin internet.

- Nombres, etiquetas y variantes de una imagen
- docker history: las capas por dentro
- docker tag: varios nombres para la misma imagen
- docker commit y por qué no usarlo
- docker save y docker load: el escenario sin internet

`labs/lab03-imagenes-y-registros/README.md`

Resumen del día

1 Por qué

Entornos idénticos, despliegues y marchas atrás triviales, versiones que conviven y todo documentado en código.

2 Qué es

Un contenedor es un proceso aislado que comparte el núcleo del sistema. Una imagen es su plantilla de solo lectura, hecha de capas.

3 Cómo se maneja

run, ps, logs, exec, inspect, stop, start, rm. Un contenedor vive mientras vive su proceso principal.

4 La regla de oro

Los contenedores son desechables. Lo que perdura va en la imagen (Dockerfile, mañana) o en un volumen (pasado mañana).

MAÑANA · DÍA 2

Dockerfile y aplicaciones .NET

Construiremos nuestras propias imágenes y contenerizaremos una API ASP.NET Core.
¿Preguntas?

